



《AI大模型Ragent项目》——知识库文档管理接口

来自： 拿个offer-开源&项目实战



马丁

2026年04月02日 22:11

前面几篇文章把文档的上传和分块处理讲完了：upload 接口负责把文件存到对象存储、元数据存到数据库，状态设为 PENDING；startChunk 接口通过 MQ 异步触发分块处理，把文档切分成 chunk 并写入向量库。这两个接口构成了文档从入库到可检索的完整链路。文档入库之后，还需要一套管理接口来控制文档的生命周期：删除不需要的文档、更新文档配置、临时下线某个文档等。这篇文章聚焦三个事务型管理接口：delete（删除文档）、update（更新文档信息）、enable（启用/禁用文档）。这三个接口看起来简单，但背后涉及分布式系统的数据一致性、事务边界设计、幂等性保障等核心问题。

三个接口概览

在深入每个接口之前，先用一张表格横向对比它们的关键特征：

维度	delete	update	enable
HTTP 方法	DELETE	PUT	PATCH
接口路径	/knowledge-base/docs/{doc-id}	/knowledge-base/docs/{docId}	/knowledge-base/docs/{docId}/enable
核心职责	删除文档及所有关联数据	更新文档信息和配置	启用/禁用文档
破坏性	高（逻辑删除）	中（配置变更）	低（状态切换）
事务范围	DB + 向量库 + 文件	DB + 调度任务	DB + chunk + 调度
幂等性	非严格幂等	部分幂等	严格幂等
关联清理	chunk/schedule/log/vector/file	无	无
调度同步	删除调度任务	同步调度配置	同步调度状态
状态校验	禁止 RUNNING	禁止 RUNNING	禁止 RUNNING

1. 共同特征

三个接口有几个明显的共同点：

- 都使用事务保护：**数据库操作（document、chunk、schedule、chunk_log 表）在 Spring 事务中，保证 ACID 特性。多表操作要么全部成功，要么全部回滚。delete 和 update 使用 @Transactional 注解声明式事务，enable 使用编程式事务（TransactionOperations）以便把耗时的 embedding 调用移到事务外，缩短事务持有时间。
- 都检查 status != RUNNING：**三个接口在执行前都会检查文档是否处于 RUNNING 状态，如果是则直接抛异常。
- 都涉及多表联动：**不只是更新 document 表，还会联动更新 chunk、schedule、chunk_log 等关联表。

1.1 为什么要检查 RUNNING 状态

分块处理是异步的，用户点击执行分块后，startChunk 接口立即返回，实际的分块任务在 MQ 消费者中执行，可能需要几分钟。如果允许在分块运行时删除或修改文档，会导致：

- 分块任务找不到文档记录：**delete 接口把文档删了，分块任务执行到一半发现文档不存在，抛异常。
- 分块任务使用旧配置：**update 接口把 chunkStrategy 从 fixed_size 改成 structure_aware，但分块任务已经开始执行，用的还是旧配置，最终写入的 chunk 和配置不一致。
- 向量库和数据库数据不一致：**分块任务正在写向量库，delete 接口把数据库记录删了，向量库里留下孤儿向量。

所以三个接口都要先检查状态，确保文档不在分块中，才能安全地执行变更操作。

1.2 事务保护的

三个接口的事务边界基本一致：

- 在事务中：**document、chunk、schedule、chunk_log 表的增删改操作，以及向量库操作（项目使用 PgVector，向量存储在 PostgreSQL 的 t_knowledge_vector 表，JdbcTemplate 自动参与 Spring 事务）。这些操作在同一个事务中，要么全部成功，要么全部回滚。

不在事务中：文件删除（deleteStoredFileQuietly），文件系统不支持事务，失败时静默处理。

需要特别说明 enable 接口：启用文档时需要调用外部 embedding 模型 API 重新计算向量，这个调用耗时较长，如果放在事务内会长时间占用数据库连接。所以 enable 接口用编程式事务，把 embedding 计算提前到事务外，只在事务内做 DB 写入和向量写入。这就引出了一个核心问题：如何保证数据库、向量库、文件系统三者的数据一致性？后面会单独拆一个大节深入讨论。

2. 差异点分析

三个接口的差异主要体现在：

破坏性程度：delete 最强（删除所有数据），update 中等（修改配置），enable 最弱（只改状态）。

幂等性设计：enable 是严格幂等（先检查当前状态），update 是部分幂等（重复更新相同内容结果不变），delete 是非严格幂等（重复删除会抛异常，但符合业务语义）。

关联数据清理：delete 需要清理 chunk、schedule、log、vector、file 五类关联数据，update 和 enable 不需要清理，只需要同步更新。

delete 接口详解

1. 接口定义

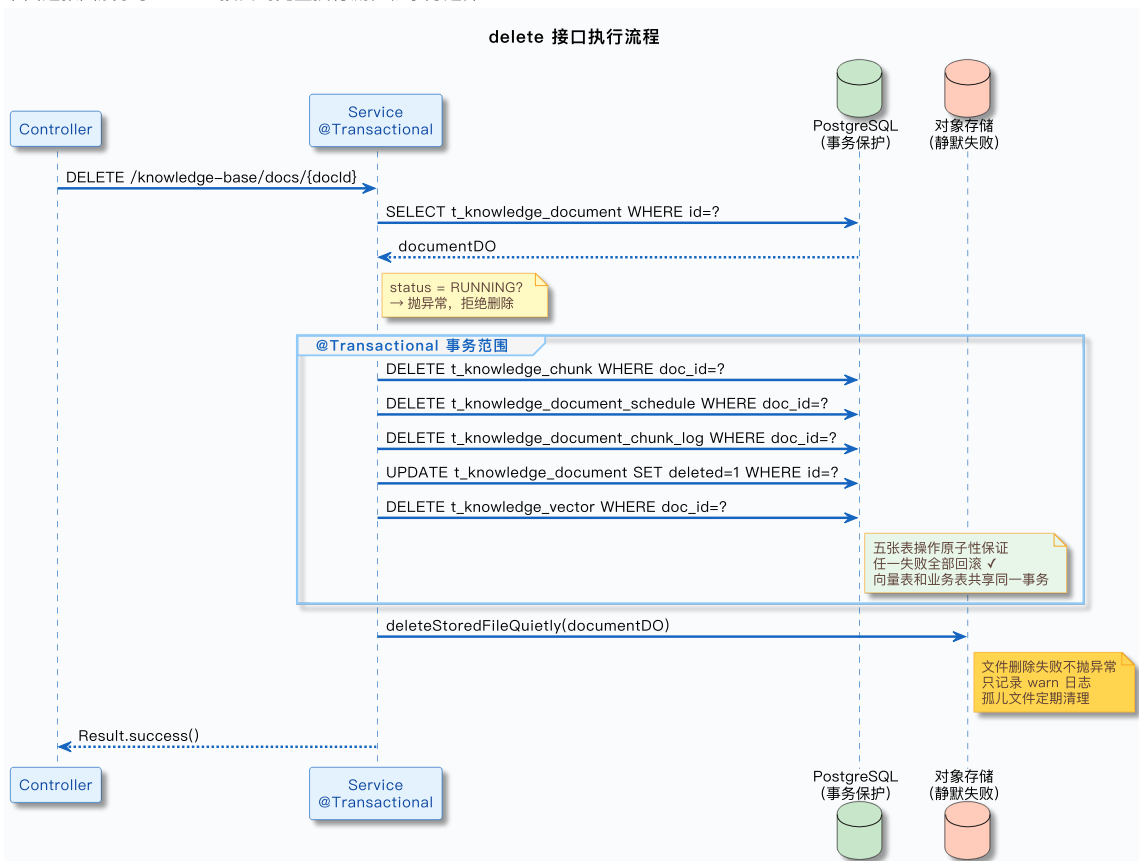
Controller 层的代码很简单：

```
@DeleteMapping("/knowledge-base/docs/{doc-id}")
public Result<Void> delete(@PathVariable(value = "doc-id") String docId) {
    documentService.delete(docId);
    return Results.success();
}
```

接口路径 DELETE /knowledge-base/docs/{doc-id}，接收一个路径参数 docId，返回 Result<Void>。注意这里用的是 DELETE 动词，符合 RESTful 规范。

2. 核心实现流程

下面这张图展示了 delete 接口的完整执行流程和事务边界：



Service 层的 delete 方法完整代码如下：

```
@Override
@Transactional(rollbackFor = Exception.class)
public void delete(String docId) {
    // 1. 查询文档记录，校验存在性
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));

    // 2. 检查文档状态，禁止删除正在分块的文档
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块中，无法删除");
    }

    // 3. 删除关联数据：chunk 记录、调度任务、分块日志
    knowledgeChunkService.deleteByDocId(docId);
    scheduleService.deleteByDocId(docId);
    chunkLogMapper.delete(Wrappers.lambdaQuery(KnowledgeDocumentChunkLogDO.class)
        .eq(KnowledgeDocumentChunkLogDO::getDocId, docId));

    // 4. 逻辑删除文档记录：设置 deleted=1
    documentDO.setDeleted(1);
    documentDO.setUpdatedBy(UserContext.getUsername());
    documentMapper.deleteById(documentDO);

    // 5. 删除向量库中的向量
    String collectionName = resolveCollectionName(documentDO.getKbId());
    vectorStoreService.deleteDocumentVectors(collectionName, docId);

    // 6. 删除存储的文件
    deleteStoredFileQuietly(documentDO);
}
```

整个流程可以分为六个步骤，下面逐个拆解。

2.1 状态校验

```
KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
Assert.notNull(documentDO, () -> new ClientException("文档不存在"));

if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
    throw new ClientException("文档正在分块中，无法删除");
}
```

和前面介绍的共同特征一样，先查文档是否存在，再检查是否 RUNNING，状态校验的原因参见 §1.1，这里不再重复。

2.2 删除关联数据

```
knowledgeChunkService.deleteByDocId(docId);
scheduleService.deleteByDocId(docId);
chunkLogMapper.delete(Wrappers.lambdaQuery(KnowledgeDocumentChunkLogDO.class)
    .eq(KnowledgeDocumentChunkLogDO::getDocId, docId));
```

按顺序删除三类关联数据：

chunk 记录：调用 `knowledgeChunkService.deleteByDocId(docId)`，删除 `t_knowledge_chunk` 表中该文档的所有分块记录。

调度任务：调用 `scheduleService.deleteByDocId(docId)`，删除 `t_knowledge_document_schedule` 表中该文档的定时同步任务（如果有）。

分块日志：直接用 MyBatis Plus 的 `delete` 方法，删除 `t_knowledge_document_chunk_log` 表中该文档的所有分块日志。

2.3 逻辑删除文档

```
documentD0.setDeleted(1);
documentD0.setUpdatedBy(UserContext.getUsername());
documentMapper.deleteById(documentD0);
```

这里用的是 MyBatis Plus 的逻辑删除机制：设置 `deleted=1`，然后调用 `deleteById`。MyBatis Plus 会自动把这个操作转换成 `UPDATE t_knowledge_document SET deleted=1 WHERE id=?`，而不是真正的 `DELETE`。

为什么是逻辑删除而不是物理删除？后面会单独讨论。

2.4 删除向量库数据

```
String collectionName = resolveCollectionName(documentD0.getKbId());
vectorStoreService.deleteDocumentVectors(collectionName, docId);
```

调用 `vectorStoreService.deleteDocumentVectors` 删除向量库中该文档的所有向量。项目使用 `PgVector`，底层是 `JdbcTemplate` 操作 `t_knowledge_vector` 表，和业务表共享同一个 Spring 事务，这个操作和前面的数据库操作是原子的。

`resolveCollectionName` 方法根据知识 ID 查询 `collection` 名称，因为向量库是按 `collection` 组织的，不同知识库的向量存在不同的 `collection` 中。

2.5 删除存储文件

```
deleteStoredFileQuietly(documentD0);
```

调用 `deleteStoredFileQuietly` 删除对象存储中的文件。方法名里的 `Quietly` 表示静默失败：如果文件删除失败（比如文件已经不存在、网络超时），不抛异常，只记录日志。

为什么要静默失败？因为文件删除失败不影响核心业务逻辑，数据库记录已经删了，向量也删了，文件留着最多占点存储空间，可以通过定时任务清理。如果文件删除失败就回滚整个事务，代价太大。

3. 设计要点

项目中用的是逻辑删除而不是物理删除，主要有四个原因：

数据审计：企业级系统需要保留操作记录，谁在什么时候删除了哪个文档，这些信息对于审计和问题排查很重要。物理删除后数据彻底消失，无法追溯。

误删恢复：用户可能手抖点错了，或者删除后发现还需要这个文档。逻辑删除可以提供恢复功能，只需要把 `deleted` 字段改回 0。物理删除后数据无法恢复。

关联数据追溯：文档删除后，`chunk`、`log` 等关联数据也删了，但如果需要排查历史问题（比如某个文档为什么分块失败了 10 次），逻辑删除可以保留这些记录。

合规要求：某些行业（金融、医疗）有数据保留期的合规要求，必须保留一定时间的历史数据，不能物理删除。

当然，逻辑删除也有缺点：数据库会越来越大，查询时需要加 `deleted=0` 条件。项目中可以通过定时任务归档或物理删除超过保留期的数据。

其实这里也可以用编程式事务来写，将最后一步 `deleteStoredFileQuietly` 放到事务外，不过呢考虑到删除本身没那么耗时，我也就使用了声明式事务注解做了。包括下面的 `update` 也是一样的逻辑。

update 接口详解

1. 接口定义

Controller 层代码：

```
@PutMapping("/knowledge-base/docs/{docId}")
public Result<Void> update(@PathVariable String docId,
                           @RequestBody KnowledgeDocumentUpdateRequest requestParam) {
    documentService.update(docId, requestParam);
    return Results.success();
}
```

接口路径 `PUT /knowledge-base/docs/{docId}`，用 `PUT` 动词表示完整更新（虽然实际是部分更新，但语义上是更新整个资源）。请求体是 `KnowledgeDocumentUpdateRequest`，用 `@RequestBody` 接收 `JSON` 格式的参数。

2. 请求参数设计

`KnowledgeDocumentUpdateRequest` 的字段设计：

```
@Data
public class KnowledgeDocumentUpdateRequest {

    /** 文档名称（必填） */
    private String docName;

    /** 处理模式：chunk / pipeline（可选） */
    private String processMode;

    /** 分块策略：fixed_size / structure_aware（processMode=chunk 时有效） */
    private String chunkStrategy;

    /** 分块参数 JSON（processMode=chunk 时有效） */
    private String chunkConfig;

    /** Pipeline ID（processMode=pipeline 时有效） */
    private String pipelineId;

    /** 来源地址（仅 URL 类型文档可更新） */
    private String sourceLocation;

    /** 是否开启定时拉取（仅 URL 类型文档可更新） */
    private Integer scheduleEnabled;

    /** 定时表达式（仅 URL 类型文档可更新） */
    private String scheduleCron;
}
```

字段分为三组：

- 基础字段：**`docName` 是必填的，其他字段都是可选的。
- 处理模式相关：**`processMode`、`chunkStrategy`、`chunkConfig`、`pipelineId`，这四个字段是互斥的，`CHUNK` 模式用前三个，`PIPELINE` 模式用最后一个。
- 调度配置相关：**`sourceLocation`、`scheduleEnabled`、`scheduleCron`，只有 `URL` 类型的文档才能更新这些字段。

3. 核心实现流程

`Service` 层的 `update` 方法比 `delete` 复杂，因为涉及字段级更新和配置校验。完整代码比较长，我们分段看：

3.1 状态校验

```
@Override
@Transactional(rollbackFor = Exception.class)
public void update(String docId, KnowledgeDocumentUpdateRequest requestParam) {
    // 1. 查询文档记录，校验存在性
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));

    // 2. 检查文档状态，禁止修改正在分块的文档
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块中，无法修改");
    }

    // ... 后续逻辑
}
```

逻辑和 delete 接口一样，这里不再重复。

3.2 文档名称更新

```
// 3. 校验并更新文档名称（必填字段）
String docName = requestParam == null ? null : requestParam.getDocName();
if (!StringUtils.hasText(docName)) {
    throw new ClientException("文档名称不能为空");
}

LambdaUpdateWrapper<KnowledgeDocumentDO> updateWrapper = Wrappers.lambdaUpdate(KnowledgeDocumentDO
    .eq(KnowledgeDocumentDO::getId, documentDO.getId())
    .set(KnowledgeDocumentDO::getDocName, docName.trim())
    .set(KnowledgeDocumentDO::getUpdatedBy, UserContext.getUsername());
```

文档名称是必填的，如果为空直接抛异常。然后用 MyBatis Plus 的 LambdaUpdateWrapper 构建更新语句，只更新 docName 和 updatedBy 两个字段。

3.3 处理模式更新

```
// 4. 如果传了 processMode，校验并更新处理配置
if (StringUtils.hasText(requestParam.getProcessMode())) {
    ProcessMode processMode = ProcessMode.normalize(requestParam.getProcessMode());
    updateWrapper.set(KnowledgeDocumentDO::getProcessMode, processMode.getValue());

    if (ProcessMode.CHUNK == processMode) {
        // CHUNK 模式：更新 chunkStrategy + chunkConfig，清空 pipelineId
        ChunkingMode chunkingMode = ChunkingMode.fromValue(requestParam.getChunkStrategy());
        String chunkConfig = validateAndNormalizeChunkConfig(chunkingMode, requestParam.getChunkCo
        updateWrapper.set(KnowledgeDocumentDO::getChunkStrategy, chunkingMode.getValue());
        updateWrapper.set(KnowledgeDocumentDO::getChunkConfig, chunkConfig);
        updateWrapper.set(KnowledgeDocumentDO::getPipelineId, null);
    } else {
        // PIPELINE 模式：更新 pipelineId，清空 chunkStrategy + chunkConfig
        if (!StringUtils.hasText(requestParam.getPipelineId())) {
            throw new ClientException("使用Pipeline模式时，必须指定Pipeline ID");
        }
        try {
            ingestionPipelineService.get(requestParam.getPipelineId());
        } catch (Exception e) {
            throw new ClientException("指定的Pipeline不存在: " + requestParam.getPipelineId());
        }
        updateWrapper.set(KnowledgeDocumentDO::getPipelineId, requestParam.getPipelineId());
        updateWrapper.set(KnowledgeDocumentDO::getChunkStrategy, null);
        updateWrapper.set(KnowledgeDocumentDO::getChunkConfig, null);
    }
}
```

处理模式更新是 update 接口最复杂的部分，因为涉及字段互斥关系。

3.3.1 CHUNK 模式配置

如果切换到 CHUNK 模式，需要做三件事：

更新 chunkStrategy：从 requestParam.getChunkStrategy() 解析出 ChunkingMode 枚举值（fixed_size 或 structure_aware），写入 chunkStrategy 字段。

更新 chunkConfig：调用 validateAndNormalizeChunkConfig 校验并规范化 chunkConfig JSON 字符串，确保格式正确、参数合法。比如 fixed_size 模式需要 chunkSize 和 overlap 参数，structure_aware 模式需要 maxChunkSize 参数。

清空 pipelineId：因为 CHUNK 模式不需要 Pipeline，所以把 pipelineId 设为 null，避免字段冲突。

3.3.2 PIPELINE 模式配置

如果切换到 PIPELINE 模式，需要做三件事：

校验 pipelineId: 必须传入 pipelineId, 否则抛异常。然后调用 `ingestionPipelineService.get(pipelineId)` 查询 Pipeline 是否存在, 如果不存在抛异常。

更新 pipelineId: 把 pipelineId 写入数据库。

清空 chunkStrategy 和 chunkConfig: 因为 PIPELINE 模式不需要这两个字段, 所以都设为 null。

这种字段互斥的设计保证了配置的完整性: CHUNK 模式有 chunkStrategy + chunkConfig, PIPELINE 模式有 pipelineId, 两者不会同时存在。

3.4 调度配置更新

```
// 5. 处理定时调度相关字段 (仅 URL 类型文档支持)
boolean scheduleChanged = false;
if (SourceType.URL.getValue().equalsIgnoreCase(documentD0.getSourceType())) {
    String newSourceLocation = requestParam.getSourceLocation();
    Integer newScheduleEnabled = requestParam.getScheduleEnabled();
    String newScheduleCron = requestParam.getScheduleCron();

    if (StringUtils.hasText(newSourceLocation)) {
        updateWrapper.set(KnowledgeDocumentD0::getSourceLocation, newSourceLocation.trim());
        scheduleChanged = true;
    }
    if (newScheduleEnabled != null) {
        updateWrapper.set(KnowledgeDocumentD0::getScheduleEnabled, newScheduleEnabled);
        scheduleChanged = true;
    }
    if (StringUtils.hasText(newScheduleCron)) {
        try {
            CronScheduleHelper.nextRunTime(newScheduleCron, new Date());
            // 验证 cron 周期不能太短 (与 upsertSchedule 保持一致)
            if (CronScheduleHelper.isIntervalLessThan(newScheduleCron, new Date(), 60)) {
                throw new ClientException("定时周期不能小于 60 秒");
            }
        } catch (IllegalArgumentException e) {
            throw new ClientException("定时表达式不合法");
        }
        updateWrapper.set(KnowledgeDocumentD0::getScheduleCron, newScheduleCron.trim());
        scheduleChanged = true;
    }
}
```

调度配置只有 URL 类型的文档才能更新, 因为只有 URL 文档支持定时拉取。

sourceLocation: URL 地址, 可以修改为新的 URL。

scheduleEnabled: 是否开启定时拉取, 0=关闭, 1=开启。

scheduleCron: 定时表达式, 比如 `0 0 2 * * ?` 表示每天凌晨 2 点执行。更新时需要校验两点:

cron 表达式格式是否合法 (调用 `CronScheduleHelper.nextRunTime` 解析, 如果抛异常说明不合法)

cron 周期是否太短 (不能小于 60 秒, 避免频繁拉取)

scheduleChanged 标记用于后续判断是否需要同步调度任务。

3.5 同步调度任务

```
// 6. 执行更新
documentMapper.update(null, updateWrapper);

// 7. 如果调度配置有变化, 同步更新调度任务
if (scheduleChanged) {
    KnowledgeDocumentD0 updatedDoc = documentMapper.selectById(docId);
    scheduleService.upsertSchedule(updatedDoc);
}
```

先执行 `documentMapper.update` 提交数据库更新，然后判断是否需要同步调度任务。

如果 `scheduleChanged=true`（说明更新了 `sourceLocation`、`scheduleEnabled` 或 `scheduleCron`），需要重新查询文档记录，然后调用 `scheduleService.upsertSchedule` 创建或更新调度任务。

注意这里用的是 `upsertSchedule` 而不是 `syncScheduleIfExists`，两者的区别：

- `upsertSchedule`：不存在则创建，存在则更新——适合 `update` 接口，因为用户可能是第一次配置定时同步
 - `syncScheduleIfExists`：只更新已有调度任务，不创建新的——适合 `enable` 接口，因为 `enable` 只改启用状态，不应该凭空创建调度任务
- 这样保证调度任务的状态与文档配置一致。

4. 设计要点

4.1 处理模式切换的字段互斥

CHUNK 模式和 PIPELINE 模式的字段是互斥的：

- CHUNK 模式**：需要 `chunkStrategy` + `chunkConfig`，不需要 `pipelineId`
- PIPELINE 模式**：需要 `pipelineId`，不需要 `chunkStrategy` + `chunkConfig`

切换模式时，必须清空互斥字段，否则会导致配置混乱。比如从 CHUNK 切换到 PIPELINE，如果不清空 `chunkStrategy`，下次切回 CHUNK 模式时，系统不知道该用新配置还是旧配置。

这种设计保证了配置的完整性和一致性：任何时候，文档的处理配置都是完整且唯一的。

4.2 为什么不自动触发重新分块

`update` 接口只更新文档配置，不会自动触发重新分块。用户如果想用新配置重新分块，需要手动调用 `startChunk` 接口。

为什么不自动触发？有三个原因：

- 职责分离**：`update` 接口的职责是更新配置，`startChunk` 接口的职责是触发分块。如果 `update` 自动触发分块，职责就混乱了。
- 用户可能只想改名称**：用户可能只是想改个文档名称，不想重新分块。如果自动触发，用户会很困惑：我只是改了个名字，为什么文档又开始分块了？
- 避免意外的耗时操作**：分块是耗时操作，可能需要几分钟。如果 `update` 接口自动触发分块，用户会觉得接口很慢，体验不好。

所以 `update` 接口只负责更新配置，是否重新分块由用户决定。这样设计更灵活，也更符合用户预期。

enable 接口详解

1. 接口定义

Controller 层代码：

```
@PatchMapping("/knowledge-base/docs/{docId}/enable")
public Result<Void> enable(@PathVariable String docId,
    @RequestParam("value") boolean enabled) {
    documentService.enable(docId, enabled);
    return Results.success();
}
```

接口路径 `PATCH /knowledge-base/docs/{docId}/enable`，用 `PATCH` 动词表示部分更新。查询参数 `value` 是布尔类型，`true` 表示启用，`false` 表示禁用。

2. 核心实现流程

Service 层的 `enable` 方法逻辑比看起来要重，不只是改个状态字段：

```
@Override
public void enable(String docId, boolean enabled) {
    // 1. 查询文档记录，校验存在性
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));

    // 2. 检查文档状态，禁止修改正在分块的文档
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块中，无法修改");
    }
}
```



```

    }

    // 3. 幂等性检查：如果已经是目标状态，直接返回
    int targetEnabled = enabled ? 1 : 0;
    if (documentD0.getEnabled() != null && documentD0.getEnabled() == targetEnabled) {
        return;
    }

    // 4. 提前查知识库，两个分支都需要，避免重复查询
    KnowledgeBaseD0 kbD0 = knowledgeBaseMapper.selectById(documentD0.getKbId());
    String collectionName = kbD0.getCollectionName();

    // 5. 启用时：embed 耗时较长，在事务外提前执行，避免长事务占用连接
    List<VectorChunk> vectorChunks = null;
    if (enabled) {
        List<KnowledgeChunkV0> chunks = knowledgeChunkService.listByDocId(docId);
        vectorChunks = chunks.stream().map(each ->
            VectorChunk.builder()
                .chunkId(each.getId())
                .content(each.getContent())
                .index(each.getChunkIndex())
                .build()
        ).toList();
        if (CollUtil.isEmpty(vectorChunks)) {
            log.warn("启用文档时未找到任何 Chunk，跳过向量重建，docId={}", docId);
            return;
        }
        chunkEmbeddingService.embed(vectorChunks, kbD0.getEmbeddingModel());
    }

    // 6. 编程式事务：DB 操作 + 向量库操作（同为 PG）一并纳入，事务尽量短
    final List<VectorChunk> finalVectorChunks = vectorChunks;
    transactionOperations.executeWithoutResult(status -> {
        documentD0.setEnabled(targetEnabled);
        documentD0.setUpdatedBy(UserContext.getUsername());
        documentMapper.updateById(documentD0);
        scheduleService.syncScheduleIfExists(documentD0);
        knowledgeChunkService.updateEnabledByDocId(docId, String.valueOf(kbD0.getId()), enabled);

        if (!enabled) {
            // 7a. 禁用：从向量库删除该文档的所有向量
            vectorStoreService.deleteDocumentVectors(collectionName, docId);
        } else {
            // 7b. 启用：写入事务外已计算好的向量
            vectorStoreService.indexDocumentChunks(collectionName, docId, finalVectorChunks);
        }
    });
}

```

整个流程可以分为七个步骤（禁用和启用分两条路径），下面逐个拆解。

2.1 状态校验

```

KnowledgeDocumentD0 documentD0 = documentMapper.selectById(docId);
Assert.notNull(documentD0, () -> new ClientException("文档不存在"));

if (DocumentStatus.RUNNING.getCode().equals(documentD0.getStatus())) {
    throw new ClientException("文档正在分块中，无法修改");
}

```

同前两个接口，这里不再重复。

```
int targetEnabled = enabled ? 1 : 0;
if (documentD0.getEnabled() != null && documentD0.getEnabled() == targetEnabled) {
    return;
}
```

先检查当前状态，如果已经是目标状态，直接返回，不执行后续操作。

这样设计**避免重复操作**：如果文档已经是启用状态，再次调用启用接口，不会执行数据库更新、调度任务同步、chunk 表更新等操作，节省资源。

不过还是要配合防重复提交来做，因为这种规避不了同时并发执行。防重复提交后面会单独说。

2.2 查询知识库配置

```
KnowledgeBaseD0 kbD0 = knowledgeBaseMapper.selectById(documentD0.getKbId());
String collectionName = kbD0.getCollectionName();
```

提前查好知识库，禁用和启用两个分支都需要 collectionName，避免重复查询。

2.3 启用时：事务外计算 embedding

```
List<VectorChunk> vectorChunks = null;
if (enabled) {
    List<KnowledgeChunkV0> chunks = knowledgeChunkService.listByDocId(docId);
    vectorChunks = chunks.stream().map(each ->
        VectorChunk.builder()
            .chunkId(each.getId())
            .content(each.getContent())
            .index(each.getChunkIndex())
            .build()
    ).toList();
    if (CollUtil.isEmpty(vectorChunks)) {
        log.warn("启用文档时未找到任何 Chunk，跳过向量重建，docId={}", docId);
        return;
    }
    chunkEmbeddingService.embed(vectorChunks, kbD0.getEmbeddingModel());
}
```

这是 enable 接口最关键的优化点。chunkEmbeddingService.embed 会调用外部 embedding 模型 API，对于内容多的文档可能需要十几秒。如果放在事务内执行，数据库连接会被长时间占用，在高并发场景下容易打满连接池。

所以把 embedding 提前到事务外执行：先从数据库读出 chunk 内容，调 API 计算好向量，存在内存里，再开事务一次性写入。事务持有时间从十几秒降低到几十毫秒。

如果 vectorChunks 为空（文档还没有分块内容），提前返回，不需要做后续操作。

2.4 编程式事务：DB + 向量库原子提交

```
final List<VectorChunk> finalVectorChunks = vectorChunks;
transactionOperations.executeWithoutResult(status -> {
    documentD0.setEnabled(targetEnabled);
    documentD0.setUpdatedBy(UserContext.getUsername());
    documentMapper.updateById(documentD0);
    scheduleService.syncScheduleIfExists(documentD0);
    knowledgeChunkService.updateEnabledByDocId(docId, String.valueOf(kbD0.getId()), enabled);

    if (!enabled) {
        vectorStoreService.deleteDocumentVectors(collectionName, docId);
    } else {
        vectorStoreService.indexDocumentChunks(collectionName, docId, finalVectorChunks);
    }
});
```

用 TransactionOperations（Spring 注入的编程式事务）开启事务，把以下操作原子提交：

- 更新 document 表的 enabled 字段
- 同步调度任务状态（syncScheduleIfExists）：文档禁用时暂停调度任务，启用时恢复
- 批量更新 chunk 表的 enabled 字段：保证检索时 chunk 状态与文档状态一致
- 向量库写入或删除：PgVector 基于 PostgreSQL，JdbcTemplate 自动加入当前事务，与 DB 操作强一致

为什么要删向量？因为检索时是直接查向量库的，如果只在数据库里禁用，向量库里的向量仍然存在，检索时还是会命中这个文档的内容，禁用就没有效果了。

还有一种设计，那就是可以在向量记录的元数据中，加个字段 enabled，停用的话可以改下状态。但是这样每次查询都要带上查询条件，感觉没必要为了这个加个必查元数据。

这里有个隐含假设：chunk 记录还在（禁用时只删向量，没有删 chunk 记录）。所以启用时可以直接从数据库读 chunk 内容，重新计算向量并写回。

3. 设计要点

3.1 级联更新

enable 接口不只是更新 document 表，会级联更新三个地方：

- 调度任务**：文档禁用后，调度任务也应该暂停，避免继续拉取数据。文档启用后，调度任务也应该恢复。
- Chunk 表**：文档禁用后，它的所有 chunk 也应该禁用，不参与检索。文档启用后，chunk 也应该启用。
- 向量库**：这是最重要的一个，也是最容易被忽视的：
 - 禁用时删除向量：检索是直接查向量库的，不删向量的话禁用就没有效果
 - 启用时重建向量索引：禁用时删了向量，启用时必须重新计算 embedding 并写回向量库

这三层级联全部在同一个编程式事务内完成（PgVector 底层是 PostgreSQL，向量操作和 DB 操作共享同一连接），任何一步失败都会整体回滚，保证一致性。

3.2 应用场景

enable 接口的典型应用场景：

- 临时下线文档**：某个文档内容有问题，需要临时下线，但不想删除。禁用后，用户检索时不会看到这个文档的内容，修复后再启用。
- A/B 测试**：想测试不同文档集合的检索效果，可以先禁用一部分文档，测试完再启用。
- 定时文档控制**：某些文档只在特定时间段有效（比如促销活动文档），可以通过定时任务调用 enable 接口控制启用/禁用。

相比 delete 接口，enable 接口更轻量，不会删除数据，可以随时恢复，适合临时性的状态控制。

小结

这篇文章深入讲解了知识库的三个事务型管理接口：delete、update、enable。

- 三个接口的共同点**：都使用事务保护数据库操作（delete 和 update 用注解式事务，enable 用编程式事务），都检查 status != RUNNING 避免并发冲突，都涉及多表联动保证数据一致性。
- 三个接口的差异点**：delete 破坏性最强，需要清理所有关联数据；update 复杂度最高，支持字段级更新和配置切换；enable 看起来轻量，实际上隐藏着向量库联动——禁用时删向量，启用时重建向量索引。
- 事务边界设计**：数据库操作和向量库操作（PgVector）在 Spring 事务中保证强一致性，文件删除在事务外静默失败接受最终一致性。enable 接口用编程式事务，把耗时的 embedding 计算提到事务外，大幅缩短事务持有时间。
- 数据一致性保障**：通过事务保护、幂等性设计、补偿机制三层保障，确保系统在各种失败场景下都能保持数据一致性。向量库操作失败会产生孤儿向量，通过定时任务清理和检索时过滤兜底。

这三个接口看起来简单，但背后涉及分布式系统的核心问题：事务边界、数据一致性、幂等性、补偿机制。理解这些设计思想，对于构建企业级系统很有帮助。

前面几篇文章讲了文档的上传、分块、定时同步，这篇讲了文档的删除、更新、启用/禁用，文档的完整生命周期管理就讲完了。至于剩下的文档查询相关接口，不涉及复杂操作，大家简单看下即可。

